

R102 - TP4 : Scapy - Réponses condensées

Premiers pas avec Scapy

Q1 : Préparation de l'environnement

But : Créer un répertoire de travail et lancer Python3 en root

```
mkdir R102TP4
cd R102TP4
sudo python3
```

Q2 : Création d'un paquet ICMP

But : Créer et afficher un premier paquet avec encapsulation Ether/IP/ICMP

```
>>> from scapy.all import *
>>> p = Ether()/IP()/ICMP()
>>> p
```

Q3 : Méthodes d'affichage

But : Comparer les différentes façons d'afficher un paquet

```
>>> p.summary()      # Résumé court
>>> p.show()         # Affichage détaillé de tous les champs
>>> ls(p)            # Liste tous les champs possibles
```

Q4 : Valeurs par défaut

But : Identifier les valeurs que Scapy met automatiquement

```
>>> ls(Ether)        # Voir champs Ethernet
>>> ls(IP)           # Voir champs IP
>>> ls(ICMP)         # Voir champs ICMP
```

Valeurs par défaut :

- Ether : dst=ff:ff:ff:ff:ff:ff, src=auto, type=0x0800
- IP : src=auto, dst=127.0.0.1, ttl=64, proto=1
- ICMP : type=8 (echo-request), code=0

Q5 : Hexdump

But : Obtenir le dump hexadécimal pour analyse détaillée

```
>>> hexdump(p)
```

Copier dans <https://hpd.gasmi.net/>

Q6 : Export PCAP

But : Sauvegarder le paquet dans un fichier pour Wireshark

```
>>> wrpcap("p.pcap", p)
```

Ouvrir avec Wireshark

Q7 : Import PCAP

But : Lire un fichier pcap existant avec Scapy

```
>>> p = rdpcap("icmp.pcap")
>>> p                                # Voir nombre de trames
>>> p.show()                        # Afficher toutes les trames
```

Q8 : Examiner une trame spécifique

But : Accéder à une trame particulière du pcap

```
>>> p[0].show()                    # Première trame (index 0)
```

Q9 : Slicing

But : Afficher plusieurs trames d'un coup

```
>>> p[0:3].show()                  # 3 premières trames
```

Q10 : Ping vers passerelle

But : Créer un ping ciblé et voir l'auto-complétion des champs

```
>>> p = Ether()/IP(dst='192.31.25.1')/ICMP()
>>> p.show()                        # Vérifier que @MAC et @IP src/dst sont complétées
```

Q11 : Accès aux champs

But : Savoir lire et modifier les champs d'un paquet

```
>>> p[IP].dst                       # Lire un champ
>>> p[IP].src
>>> p[Ether].dst
>>> p[Ether].src
>>> p[ICMP].type
```

Envoi et réception

Q1 : Envoi simple sans récupération de réponse

But : Envoyer un ping sans attendre la réponse

```
# Avec couche Ether (sendp)
>>> p = Ether()/IP(dst='192.31.25.1')/ICMP()
>>> sendp(p)

# Sans couche Ether (send)
>>> p = IP(dst='192.31.25.1')/ICMP()
>>> send(p)
```

Q2 : Envoi avec récupération de la réponse

But : Envoyer un ping et capturer la réponse (Echo Reply)

```
# Avec Ether (srp1)
>>> p = Ether()/IP(dst='192.31.25.1')/ICMP()
>>> srp1(p)

# Sans Ether (sr1)
>>> p = IP(dst='192.31.25.1')/ICMP()
>>> sr1(p)

# Stocker pour analyser
>>> rep = srp1(p)
>>> rep.summary()
>>> rep.show()
```

Q3 : Cas sans réponse

But : Comprendre ce qui se passe quand une machine ne répond pas

```
>>> p = IP(dst="192.31.25.99")/ICMP()
>>> rep = sr1(p, timeout=5)
>>> print(rep) # Affiche None si pas de réponse
```

Script scanpings.py

But : Créer un scanner pour détecter les machines actives sur un réseau

```
#!/usr/bin/env python3
import argparse
import logging
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
from scapy.all import *

parser = argparse.ArgumentParser(description="Scan ping")
```

```

parser.add_argument("max", type=int, help="Valeur du dernier octet max des @IP
a pinguer")
args = parser.parse_args()

for i in range(1, args.max + 1):
    ip = '192.31.25.' + str(i)
    p = IP(dst=ip)/ICMP()
    conf.verb = 0
    rep = sr1(p, timeout=5)
    if rep != None:
        print(f"{ip} repond (host up)")
    else:
        print(f"{ip} ne repond pas (host down)")

```

Utilisation :

```

chmod +x scanpings.py
sudo ./scanpings.py 5

```

Requête ARP

Q1 : Encapsulation ARP

But : Rappeler comment ARP est encapsulé **Réponse :** ETH | ARP (pas d'IP, ARP est en couche 2)

Q2 : Test requête ARP Who-has

But : Envoyer une requête ARP pour obtenir l'@MAC de la passerelle

```

>>> p = Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(pdst='192.31.25.1')
>>> p.show()           # Vérifier la requête (op=who-has)
>>> rep = srp1(p)       # Envoyer et recevoir
>>> rep.summary()       # Voir "is-at"
>>> rep.show()          # Voir l'@MAC retournée

```

Vérifier avec :

```
ip n      # Table ARP système
```

Requête DNS

Q1 : Encapsulation DNS

But : Rappeler comment DNS est encapsulé **Réponse :** ETH | IP | UDP | DNS

Q2 : nslookup classique

But : Identifier le serveur DNS à interroger

```
nslookup rt-bethune.univ-artois.fr
```

Noter l'@IP du serveur DNS (ex: 172.18.26.101)

Q3 : Requête DNS avec Scapy

But : Interroger directement le serveur DNS avec Scapy

```
>>> p = IP(dst='172.18.26.101')/UDP()/DNS(qd=DNSQR(qname='rt-bethune.univ-artois.fr'))
>>> rep = sr1(p)
>>> rep.summary()      # Voir l'@IP résolue
>>> rep.show()         # Détails de la réponse DNS
```

Script scanports.py

But : Créer un scanner de ports TCP pour détecter les services actifs

Q1 : Test connexion TCP avec flag SYN

But : Vérifier si un port est ouvert (réponse SYN-ACK)

```
>>> p = IP(dst='192.31.25.1')/TCP(dport=22, flags='S')
>>> rep = sr1(p, timeout=1)
>>> rep.show()
>>> rep[TCP].flags == 'SA' # True si port ouvert
```

Q2 : Tester d'autres ports

But : Vérifier plusieurs ports courants (FTP, Telnet, HTTP)

```
>>> p = IP(dst='192.31.25.1')/TCP(dport=20, flags='S') # FTP data
>>> rep = sr1(p, timeout=1)

>>> p = IP(dst='192.31.25.1')/TCP(dport=23, flags='S') # Telnet
>>> rep = sr1(p, timeout=1)

>>> p = IP(dst='192.31.25.1')/TCP(dport=80, flags='S') # HTTP
>>> rep = sr1(p, timeout=1)
```

Q3 : Script complet

But : Automatiser le scan de plusieurs ports

```
#!/usr/bin/env python3
import argparse
import logging
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
from scapy.all import *
```

```

parser = argparse.ArgumentParser(description="Scan ports")
parser.add_argument("ip", type=str, help="@IP a scanner")
args = parser.parse_args()

PORTS_A_SCANNER = [20, 22, 23, 80]

for i in PORTS_A_SCANNER:
    conf.verb = 0
    p = IP(dst=args.ip)/TCP(dport=i, flags='S')
    rep = sr1(p, timeout=1)
    if rep != None and rep.haslayer(TCP) and rep[TCP].flags == 'SA':
        print(f"Port {i} ouvert sur {args.ip}")

```

Utilisation :

```

chmod +x scanports.py
sudo ./scanports.py 192.31.25.1

```

Aide-mémoire

Fonctions d'envoi

Fonction	Couche	Retour	Utilité
send()	IP	Aucun	Envoi simple couche 3
sendp()	Ether	Aucun	Envoi simple couche 2
sr1()	IP	1ère réponse	Envoi + réception couche 3
srp1()	Ether	1ère réponse	Envoi + réception couche 2

Encapsulations courantes

IP()/ICMP()	# Ping
Ether()/ARP(pdst='...')	# ARP Who-has
IP()/TCP(dport=X, flags='S')	# TCP SYN scan
IP()/UDP()/DNS(qd=DNSQR(qname='...'))	# Requête DNS

Préparation et environnement

- **sudo python3** : Lance Python avec les droits root, indispensables pour envoyer/capturer des paquets réseau.
- **mkdir / cd** : Crée et organise le répertoire de travail du TP.

Création et manipulation de paquets

- **Ether()/IP()/ICMP()** : Construit un paquet réseau en empilant les couches (Ethernet, IP, ICMP).
- **/ (opérateur Scapy)** : Sert à encapsuler/empiler les couches réseau.

Affichage et analyse

- **p.summary()** : Affiche un résumé très court du paquet.
- **p.show()** : Affiche tous les champs du paquet de façon détaillée.
- **ls(p)** : Liste tous les champs disponibles d'un paquet ou d'une couche.
- **hexdump(p)** : Affiche le paquet en hexadécimal pour une analyse bas niveau.

Valeurs par défaut

- **ls(Ether) / ls(IP) / ls(ICMP)** : Montre les champs et valeurs par défaut automatiquement définis par Scapy.

PCAP (Wireshark)

- **wrpcap("file.pcap", p)** : Enregistre un paquet ou une capture dans un fichier pcap.
- **rdpcap("file.pcap")** : Lit un fichier pcap et le charge dans Scapy.
- **p[i]** : Accède à une trame précise dans un fichier pcap.
- **p[x:y]** : Affiche plusieurs trames à la fois (slicing).

Accès aux champs

- **p[IP].src / p[IP].dst** : Lit les adresses IP source et destination.
- **p[Ether].src / p[Ether].dst** : Lit les adresses MAC source et destination.
- **p[ICMP].type** : Lit ou modifie le type ICMP (ex : echo-request).

Envoi de paquets

- **send()** : Envoie un paquet en couche 3 (IP), sans attendre de réponse.
- **sendp()** : Envoie un paquet en couche 2 (Ethernet), sans attendre de réponse.

Envoi avec réponse

- **sr1()** : Envoie un paquet IP et récupère la première réponse.
- **srp1()** : Envoie un paquet Ethernet et récupère la première réponse.
- **timeout=X** : Évite d'attendre indéfiniment une réponse.
- **rep != None** : Vérifie si une réponse a été reçue.

ICMP (Ping)

- **IP(dst="X")/ICMP()** : Crée un ping ICMP vers une machine cible.

ARP

- **Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(pdst='X')** : Envoie une requête ARP en broadcast pour connaître une adresse MAC.
- **ip n** : Affiche la table ARP du système.

DNS

- **IP()/UDP()/DNS(qd=DNSQR(qname='nom'))** : Crée une requête DNS manuelle avec Scapy.
- **nslookup nom** : Identifie le serveur DNS à interroger.

TCP / Scan de ports

- **IP()/TCP(dport=X, flags='S')** : Envoie un SYN pour tester si un port TCP est ouvert.
- **rep[TCP].flags == 'SA'** : Indique que le port est ouvert (SYN-ACK).
- **flags='RA'** : Indique que le port est fermé (RST-ACK).

Mode silencieux

- **conf.verb = 0** : Désactive les messages intermédiaires de Scapy.